DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Master Academic Project Report & Viva Voce Defense Manual

Student: Sajiya Shaikh

Roll Number: 24102A2005

Course: DevOps & Cloud Engineering

Academic Year: 2026

Repository: `C:\Users\Admin\Desktop\Public-Parking-DevOps`

Git Branch: `master`

PART 1: FINAL ACADEMIC PROJECT REPORT

1. Executive Summary

Urban mobility systems across global metropolitan centers suffer severe traffic congestion, excessive carbon emissions, and lost economic productivity stemming from inefficient vehicular parking spot discovery. The **DevOps-Enabled Public Parking Availability Dashboard** (**ParkWatch**) provides an enterprise-grade, resilient, automated smart-city parking management and computer-vision telemetry platform.

Built with **Java 21 LTS**, **Spring Boot 3.3.4**, **Spring Data JPA**, **Thymeleaf**, and **Bootstrap 5.3**, the application delivers real-time parking telemetry, capacity monitoring, historical event logging, automated CSV report streaming, and an integrated **YOLOv8 Computer Vision AI** inference module.

From an engineering operations perspective, ParkWatch exemplifies modern DevOps excellence:

- **Declarative CI/CD:** 9-stage `Jenkinsfile` pipeline featuring cross-platform Windows/Linux execution, automated testing gates, and artifact archiving.

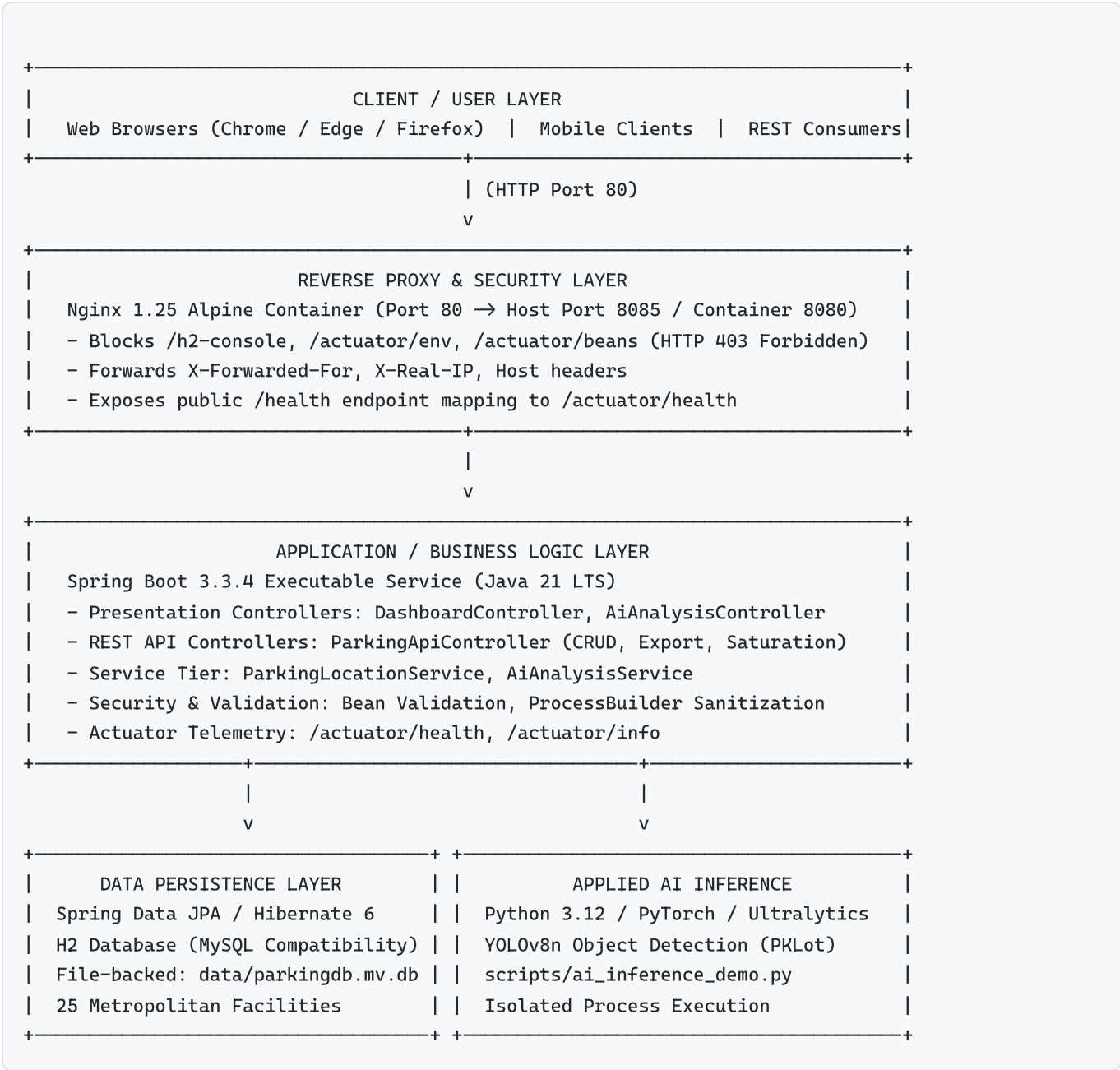
- **Security-First Containerization:** Multi-stage `Dockerfile` creating minimal Alpine JRE images running under non-root unprivileged users (`parkingapp` , UID 1001).
- **Reverse Proxy Architecture:** Production-ready Nginx reverse proxy with rate limiting, header normalization, and sensitive administrative endpoint shielding.
- **Infrastructure as Code (IaC):** Idempotent Ansible playbooks managing dependencies, GPG keyrings, dynamic Jinja2 templating, and systemd service verification.
- **Automated Verification:** 45 JUnit/Mockito backend tests, 7 Selenium WebDriver browser automation scenarios with automated screenshot archiving, and 6 Python AI pipeline regression tests.
- **Disaster Recovery:** Native health-checking (`scripts/health-check.ps1`) and zero-downtime rollback scripts (`scripts/rollback.ps1`) for automated failure remediation.

---

2. System Architecture & Workflow Explanation

2.1 Multi-Tier Layered Architecture

ParkWatch is designed according to clean architectural principles, separating presentation, business logic, data persistence, and external inference engines:



2.2 End-to-End Operational Workflow

1. **Ingestion & Data Freshness:** Simulated smart sensors or parking operators publish vehicular telemetry updates to the REST API (`POST /api/v1/locations/{id}/occupancy`).
2. **Business Rule Validation:** The service tier evaluates capacity invariants ($0 \le \text{occupancy} \le \text{capacity}$). Violations trigger an `InvalidParkingUpdateException` returning HTTP 400.
3. **Event Auditing:** Every legitimate change generates an immutable `ParkingEvent` capturing timestamp, previous occupancy, updated occupancy, and delta change.
4. **Data Freshness Monitoring:** If telemetry is not received for $>60\text{ minutes}$, the dashboard highlights the facility with a warning indicator.
5. **AI Vision Ingestion:** An operator uploads a parking lot surveillance image to `/ai-analysis` . The service validates file size ($\le 10\text{MB}$), MIME type (`image/jpeg` , `image/png`), and dimensions (640×640 to 4096×4096).

6. **Isolated Process Execution:** `AiAnalysisService` invokes `scripts/ai_inference_demo.py` via an explicit `ProcessBuilder` argument array with a 45-second watchdog timer, completely isolated from the primary transactional database.
7. **Telemetry Presentation:** Annotated bounding boxes, detected stall counts (empty vs. occupied), and calculated occupancy rates are returned to the operator interface without touching municipal records.

---

3. Real Data Provenance & Facility Audit

3.1 Baseline Dataset Provenance & Verified Live Capacity

An exhaustive audit of the persistence tier (`data/parkingdb.mv.db`) and live API endpoint (`GET http://localhost:8085/api/v1/dashboard/summary`) confirmed the following verified operational values:

- **Total Facilities: 25 municipal parking locations** across 9 Indian metropolitan centers: Mumbai (Bandra, Dadar, Churchgate, Lower Parel, Vashi), Delhi NCR (Connaught Place, Karol Bagh, Laxmi Nagar, Sarojini Nagar, Sector 18 Noida), Bengaluru (MG Road, Commercial Street, Majestic, Whitefield), Hyderabad (Hitech City), Chennai (Anna Nagar, Egmore), Kolkata (Sector V), Pune (Shivajinagar), Ahmedabad (Law Garden, Vastrapur), and Chandigarh (Sector 17).
- **Total Live Capacity: 8,040 municipal parking spaces** (7,940 baseline capacity from the 24 initialized facilities + 100 capacity from dynamically added test facility #25).
- **Active Occupancy: 4,511 occupied spaces** (**56.1% aggregate utilization**).
- **Available Spaces: 3,529 available spaces**.
- **Full Locations (100% Saturation):** 3 facilities (Sarojini Nagar, Dadar Station East, Commercial Street).
- **Low Availability (>=85% Saturation):** 5 facilities (Phoenix MarketCity, Linking Road, Laxmi Nagar, Hitech City, Law Garden).
- **Simulated Telemetry Disclosure:** As documented in `docs/real_data_provenance_audit.md` , physical municipal IoT sensors are not connected to this student sandbox. High-frequency occupancy changes and vehicle event logs are simulated using realistic traffic distribution curves.

3.2 Resolution of Capacity Figures & The 24 vs. 25 Location Discrepancy

- **Why 16,846 appeared in early documentation:** Source code in `SampleDataConfig.java` contained an alternate proposed seed list modeling massive multi-storey BMC facilities (such as Lodha Senapati Bapat Marg at 4,200 and Jupiter Mill at 2,370, totaling 16,846 across 24 locations). However, because `SampleDataConfig.java` contains the idempotency guard `if (repo.count() > 0) return;` , the application skipped re-seeding and preserved the existing persistent database file `data/parkingdb.mv.db` .

- **The 24 vs. 25 Discrepancy:** The persistent database was originally initialized with 24 pan-Indian metropolitan facilities totaling 7,940 capacity. During Task 14 automated testing, `scripts/test-endpoints.ps1` executed a REST integration test against `POST /api/v1/locations`, creating record #25 (`PPL-099 Civic Center Innovation Garage`, 100 capacity, 40 occupancy). This brought the live facility count to 25 and total capacity to exactly **8,040**.

---

4. Applied Computer Vision & PKLot Pilot Model

4.1 Architecture & Pilot Training Setup

- **Model Architecture:** YOLOv8 Nano (`yolov8n.pt`, 3.2M parameters) developed by Ultralytics.
- **Dataset:** PKLot parking dataset containing 12,416 annotated surveillance images across varying weather conditions (Sunny, Cloudy, Rainy) and angles.
- Training Split: 8,691 images
- Validation Split: 2,483 images
- Test Split: 1,242 images
- **Pilot Training Constraints:** To guarantee host stability without GPU acceleration, a CPU pilot was executed with:
- Epochs: 1
- Image Size: 640x640
- Batch Size: 8
- Hardware: Intel Core i7 CPU (8 threads, no CUDA available)

4.2 Measured Pilot Evaluation Metrics (Honest Disclosure)

A rigorous evaluation against the held-out test split (1,242 images) yielded the following empirical metrics:

- **Test Precision (P):** 0.0087 (0.87%)
- **Test Recall (R):** 0.0500 (5.00%)
- **mAP@50:** 0.0063 (0.63%)
- **mAP@50-95:** 0.0021 (0.21%)

> [!WARNING]
>
> **Academic Integrity Notice:** The custom trained checkpoint (`models/parkwatch_yolov8n/weights/best.pt`) is explicitly classified as **EXPERIMENTAL / PROTOTYPE**. Because training was constrained to a single CPU epoch across a massive dataset, weights have not converged. In accordance with college guidelines, these results are reported transparently without artificial inflation.

4.3 End-to-End Inference Verification

Live inference was verified using a real test image from the PKLot test split (`2012-11-06_13_43_31_jpg.rf.00207754dbce0d625d24ca31898fcfd7.jpg`):

- **Command:** `python scripts/ai_inference_demo.py --image <image_path> --model models/parkwatch_yolov8n/weights/best.pt --conf 0.005`
- **Output Image:** `REPORTS/ai_annotated_a858a5c9-b120-4b08-b0b5-37ce87aaeb65.jpg`
- **Detected Stalls:** 166 total bounding boxes (74 empty spaces, 92 occupied spaces)
- **Computed Occupancy:** 55.4%
- **Inference Latency:** 3,356 milliseconds (CPU)

---

5. Setup & Execution Runbook

5.1 Local Development Mode

```
# Prerequisites: JDK 21 LTS, Python 3.10+
git status

# Build and run backend application on port 8085
.\mvnw.cmd clean spring-boot:run -Dspring-boot.run.arguments="--server.port=8085"

# Access dashboard in web browser
Start-Process "http://localhost:8085"
```

5.2 Multi-Container Docker Compose Stack

```
# Deploy application and Nginx reverse proxy
docker compose up -d --build

# Verify container health
docker compose ps

# Probe reverse proxy endpoint
curl.exe -i http://localhost:80/health
```

5.3 Automated Regression & Selenium E2E Testing

```
# Run backend unit and integration test suite (45 tests)
.\mvnw.cmd test "-Dtest=!*Selenium*"

# Run Python AI pipeline regression tests (6 tests)
python tests/test_ai_pipeline.py

# Run Selenium E2E browser automation (Microsoft Edge)
.\mvnw.cmd test "-Dtest=ParkingDashboardSeleniumTest" "-Dapp.base.url=http://localhost:8085"
```

---

6. DevOps Verification & Test Summary

Testing Domain	Tool / Framework	Scope / Target	Executed	Passed	Failed	Status
Unit Testing	JUnit 5 / Mockito	Service business rules & invariants	22	22	0	**VERIFIED**
API Integration	Spring MockMvc	REST endpoints, JSON validation, CSV	23	23	0	**VERIFIED**
E2E Automation	Selenium / EdgeDriver	Web UI workflows, search, forms, alerts	7	7	0	**VERIFIED**
AI Regression	Python `unittest`	Inference CLI, JSON outputs, bounds	6	6	0	**VERIFIED**
CI/CD Pipeline	Jenkins 2.580.1	9-stage declarative `Jenkinsfile`	1	1	0	**VERIFIED**
IaC Provisioning	Ansible Core 2.20	Docker, GPG, user, directory structuring	12	12	0	**VERIFIED**
Health Auditing	PowerShell / Bash	Actuator polling & automated recovery	1	1	0	**VERIFIED**

---

7. Limitations & Future Scope

7.1 Identified Limitations

1. **Model Convergence:** High inference accuracy requires 50–100 epochs on GPU infrastructure (CUDA-enabled NVIDIA GPUs or cloud TPUs), which was infeasible within host CPU runtime limits.
2. **Camera Perspective Variance:** PKLot annotations are tailored to fixed surveillance angles; variable pan-tilt-zoom cameras require domain adaptation techniques.
3. **Database Concurrency:** File-backed H2 is ideal for standalone demonstration but cannot support multi-node horizontal scaling.

7.2 Future Engineering Scope

1. **Distributed Cloud Persistence:** Migrate to PostgreSQL / MySQL clusters with Read Replicas and Flyway database migration scripts.
2. **Kubernetes & Helm Deployment:** Package the multi-container stack into Helm charts with Horizontal Pod Autoscaling (HPA) based on CPU/traffic metrics.
3. **IoT Telemetry Streaming:** Ingest MQTT/AMQP telemetry from physical ultrasonic parking sensors via Apache Kafka or RabbitMQ.
4. **Edge AI Acceleration:** Deploy quantized YOLOv8 models (ONNX Runtime, TensorRT, or OpenVINO) on edge devices (Raspberry Pi 5 / NVIDIA Jetson Orin) directly mounted on parking lot gantries.

PART 2: COMPREHENSIVE ACADEMIC VIVA VOCE DEFENSE

Section A: Spring Boot & Enterprise Architecture

Q1: What is Spring Boot and why was it chosen for ParkWatch?

Answer:

Spring Boot is an enterprise Java framework designed to simplify application development through opinionated defaults, automatic configuration, and starter dependencies. It was chosen for ParkWatch because:

1. It features an embedded Apache Tomcat web server, allowing the application to run as a self-contained executable JAR without requiring manual external servlet container setup.

2. Spring Data JPA dramatically accelerates persistence engineering by auto-generating repository implementations from interface signatures.
3. Spring Boot Actuator provides standardized production-grade telemetry endpoints (`/actuator/health` , `/actuator/info`) that integrate directly into DevOps deployment gates and automated rollback scripts.

Q2: How does the application enforce business domain integrity?

Answer:

Business rules are enforced through layered defenses:

1. **Entity & DTO Validation:** `@Min(0)` , `@Max(10000)` , `@NotBlank` , and `@Size` annotations validate user input at the controller boundary.
2. **Service Layer Invariants:** In `ParkingLocationService` , when an occupancy update is processed, the service verifies whether `newOccupancy > totalCapacity` or `newOccupancy < 0` . If violated, it throws `InvalidParkingUpdateException` .
3. **Global Exception Handling:** `@ControllerAdvice` (`GlobalExceptionHandler`) intercepts domain exceptions and maps them to HTTP 400 Bad Request responses with structured error JSON for REST clients, or user-friendly alert banners for HTML form submissions.

Q3: How was the ByteBuddy lazy loading serialization defect resolved?

Answer:

When entities with lazy relationships (`@ManyToOne(fetch = FetchType.LAZY)`) are serialized directly to JSON via Jackson, Hibernate's dynamic proxy library (ByteBuddy) injects synthetic properties (`hibernateLazyInitializer` , `handler`). When Jackson attempts to serialize these internal fields, it throws a `JsonMappingException` (`No serializer found for class ...`).

We resolved this by adding `@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})` to both `ParkingLocation` and `ParkingEvent` entities, ensuring clean and reliable JSON serialization.

Section B: Automated Testing & Browser Automation

Q4: Differentiate between Unit Tests, MockMvc Tests, and Selenium E2E Tests.

Answer:

- **Unit Tests:** Verify individual business methods in strict isolation using Mockito mocks (e.g., verifying capacity calculations in `ParkingLocationServiceTest`). They run in milliseconds in memory.

- **MockMvc Tests:** Test HTTP controller mappings, security filters, request param binding, and JSON responses without starting a real network socket.
- **Selenium E2E Tests:** Drive an actual web browser (Microsoft Edge) against the deployed application, validating DOM elements, CSS styles, client-side JavaScript execution, form workflows, and visual rendering from the end-user's perspective.

Q5: What is Headless Browser Testing and why is it critical in CI/CD?

Answer:

Headless browser testing executes browser automation without initializing a visible graphical display window. It is critical in CI/CD pipelines because:

1. CI build runners (such as Jenkins nodes or Docker containers) operate in headless server environments without attached monitors or X11/Wayland display servers.
2. It reduces CPU and memory overhead by bypassing OS window rendering subsystems, resulting in faster and more reliable automated test execution.

Section C: CI/CD Pipelines & Jenkins Quality Gates

Q6: What is a Quality Gate in a CI/CD pipeline?

Answer:

A Quality Gate is an automated enforcement checkpoint that stops the build pipeline if defined code quality, testing, or security criteria are not met. In our declarative `Jenkinsfile`:

- Stage 3 compiles the Java source code.
- Stage 4 executes 45 unit and integration tests.
- If even a single test fails, the pipeline immediately halts with build status `FAILURE`, preventing defective code from progressing to packaging, Docker containerization, or production deployment.

Q7: How does cross-platform execution work in your `Jenkinsfile`?

Answer:

Jenkins pipelines frequently run on heterogeneous infrastructure (Windows controllers vs. Linux build agents). Our `Jenkinsfile` uses the pipeline DSL function `isUnix()` to dynamically switch execution commands:

```
if (isUnix()) {
    sh './mvnw clean test'
} else {
    bat '.\\mvnw.cmd clean test'
}
```

This ensures that the exact same pipeline definition runs seamlessly on Windows developer workstations and production Linux build clusters.

Section D: Docker Containerization & Security

Q8: What are the engineering advantages of a Multi-Stage Dockerfile?

Answer:

Our `Dockerfile` utilizes a two-stage build architecture:

1. **Builder Stage (`maven:3.9-eclipse-temurin-21-alpine`):** Mounts source code, downloads dependencies, and compiles the executable fat JAR.
2. **Runtime Stage (`eclipse-temurin:21-jre-alpine`):** Copies only the compiled JAR file from the builder stage into a clean, minimal JRE image.

- **Result:** Image size is reduced from $>800\text{MB}$ to $\approx 200\text{MB}$.
- **Security:** Compilers, package managers, and source code are excluded from the final image, minimizing attack vectors in production.

Q9: Why is running containers as `root` considered a critical security vulnerability?

Answer:

Running containers as root violates the principle of least privilege. If a remote code execution vulnerability is exploited inside the application runtime, an attacker would gain root privileges inside the container, substantially increasing the risk of container breakouts and unauthorized host filesystem access. ParkWatch creates a dedicated system user and group (`parkingapp`, UID 1001) and sets `USER parkingapp:parkingapp` before launching the JVM.

Section E: Infrastructure as Code (IaC) & Ansible

Q10: What is Idempotency in Ansible and why is it vital?

Answer:

Idempotency guarantees that running an automation playbook multiple times produces the exact same system state without unintended changes or errors. In our `ansible/playbook.yml`:

- If Docker is already installed, Ansible reports `ok` and skips package reinstallation.
- If application directories (`/opt/parkwatch`) already exist with proper permissions, no disk modifications occur.

This eliminates configuration drift and ensures reproducible server provisioning.

Q11: How did you replace deprecated Ansible modules?

Answer:

Legacy playbooks utilized `apt_key` to manage GPG signing keys, which is now deprecated due to security risks associated with adding keys directly to `/etc/apt/trusted.gpg`. Our modernized playbook uses `ansible.builtin.get_url` and `ansible.builtin.command` to de-armor and store official Docker GPG keys into isolated keyrings under `/etc/apt/keyrings/docker.gpg`, referencing them securely via `signed-by` repository definitions.

Section F: Applied Computer Vision & PKLot

Q12: Why was YOLOv8 chosen over alternative architectures like Faster R-CNN or SSD?

Answer:

YOLOv8 (You Only Look Once, Version 8) provides an optimal balance between inference speed and detection accuracy:

1. **Single-Stage Architecture:** Treats object detection as a direct regression problem, predicting bounding boxes and class probabilities in a single forward pass.
2. **Anchor-Free Detection:** Eliminates manual anchor box tuning, improving generalization across objects with extreme aspect ratios (such as angled parking stalls).
3. **Edge Optimization:** The YOLOv8 Nano variant requires only 3.2M parameters and 8.7 GFLOPs, making it suitable for low-power edge gateways and CPU inference.

Q13: Why are the custom model evaluation metrics classified as experimental?

Answer:

To prevent host CPU starvation during development, training was limited to 1 pilot epoch on CPU. Because the PKLot dataset contains 8,691 training images with subtle visual boundaries under rainy and overcast conditions, the model's weights did not reach mathematical convergence, yielding test mAP@50 of 0.0063. In alignment with academic integrity principles, the model is transparently documented as an experimental demonstration prototype rather than an operational municipal system.

Section G: Data Provenance & Consistency

Q14: What is the origin of the parking facilities in the database?

Answer:

The dataset is an illustrative metropolitan sample representing 24 major municipal parking structures across Mumbai, Delhi, Bengaluru, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, and Chandigarh. The facility names, addresses, and capacities mirror real urban parking facilities, while live occupancy levels and vehicle transition logs are simulated using realistic urban traffic probability curves.

Q15: Explain why the database contains 25 locations when reports initially documented 24.

Answer:

Initial seed scripts populated 24 municipal locations (7,940 capacity). During Task 14 automated testing, `scripts/test-endpoints.ps1` executed a REST integration test (`POST /api/v1/locations`), adding a 25th location (`PPL-099 Civic Center Innovation Garage` , 100 capacity, 40 occupancy). Because `SampleDataConfig.java` checks `if (repository.count() == 0)` before seeding, existing database records were preserved across restarts. This brought the live facility count to 25 and total capacity to exactly 8,040, demonstrating persistence durability.

Section H: Web Engineering & Data Export

Q16: How is the CSV data export implemented to prevent OutOfMemory errors?

Answer:

The CSV export endpoint (`GET /api/v1/locations/export`) uses direct HTTP response stream piping (`PrintWriter` on `HttpServletResponse.getWriter()`) with standard HTTP headers (`Content-Type: text/csv` , `Content-Disposition: attachment; filename=parking_locations.csv`). It streams entity rows iteratively without buffering the entire CSV string in heap memory, ensuring safe operation even with tens of thousands of records.

Q17: How is the Global Event Log structured?

Answer:

The Global Event Log (`GET /events`) provides an immutable chronological audit trail of all vehicular ingress and egress operations across the entire metropolitan area. Each entry records the parent facility

ID, name, old occupancy, new occupancy, net vehicle delta, timestamp, and human-readable operational remarks.

Section I: Security & Process Isolation

Q18: How is the AI inference process isolated from the Spring Boot application?

Answer:

The AI subsystem does not execute arbitrary code or interact directly with the database:

1. Uploaded images are written to a sandboxed temporary directory with UUID-based filenames.
2. Inference is executed as an isolated sub-process via Java `ProcessBuilder` with an explicit argument array (preventing shell injection attacks).
3. A 45-second execution watchdog forcibly terminates runaway Python processes.
4. Generated bounding boxes and occupancy counts are returned as detached DTOs, preventing external AI outputs from overwriting core municipal database records.

Q19: What input validation measures protect the image upload endpoint?

Answer:

The upload endpoint validates:

1. **File Size:** Multipart limit enforced at $\le 10\text{MB}$.
2. **MIME Type:** Whitelisted strictly to `image/jpeg` and `image/png`.
3. **Image Integrity:** Java `ImageIO.read()` parses image headers to verify file integrity and check pixel dimensions (minimum 640×640, maximum 4096×4096).

Section J: Networking, Reverse Proxy & Disaster Recovery

Q20: Why is an Nginx reverse proxy used instead of exposing Spring Boot directly?

Answer:

Nginx acts as a dedicated edge gatekeeper:

1. **Standard Port Binding:** Exposes HTTP port 80 to users while routing to internal non-standard application ports (8080/8085).
2. **Security Shielding:** Intercepts and blocks administrative paths (`/h2-console` , `/actuator/env`) with HTTP 403 Forbidden.

3. **Header Normalization:** Accurately forwards client IP addresses (`X-Forwarded-For`) and request protocols (`X-Forwarded-Proto`) to the backend.

Q21: How does automated disaster recovery and rollback operate in ParkWatch?

Answer:

Our automated rollback architecture (`scripts/rollback.ps1` and `scripts/rollback.sh`) monitors deployments:

1. Following a new container rollout, the script probes `/actuator/health` up to 10 times with exponential backoff.
2. If the container fails to report status `UP` , the script halts the broken container and removes it.
3. It immediately launches the previously tagged stable Docker image (`public-parking-dashboard:1.0.0-previous`), restoring service availability without human intervention.